

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Technical Presentation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 2 – Technical Presentation
Weightage:	30% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P.Hitha6ishi	Reg. No.:	192425196
Section / Batch:	C/2024	Date:	16-07-2026

Code of Conduct

I P.Hithaishi (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Hithaishi

Instructions

- Suggested Team Member Roles (3 Members)
- Member 1: Theory, Concepts, and Literature Review
- Member 2: Algorithm Design, Models, and Technical Analysis
- Member 3: Demonstration, Case Study, Applications, and Conclusion
- GPS photo must submit.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Evolution of Natural Language Processing: From Rule-Based Systems to Modern NLP	NLP Models, Language Processing Evolution	TP-01
Comparative Study of NLP Models and Algorithms	Model Analysis, Algorithm Comparison	TP-02
Regular Expressions for Real-World Text Processing	Pattern Matching, Information Extraction	TP-03
Designing Efficient Regular Expression Patterns for Information Extraction	Text Mining, Data Extraction Techniques	TP-04
Finite State Automata in Natural Language Processing	State Transitions, Language Recognition	TP-05
Building a Finite State Automaton for Token Recognition	Lexical Analysis, Token Validation	TP-06
Morphology and Word Formation in Natural Languages	Morpheme Analysis, Word Structure Processing	TP-07
Inflectional Morphology: Understanding Grammatical Variations	Grammatical Analysis, Word Normalization	TP-08
Derivational Morphology and Vocabulary Expansion	Word Generation, Semantic Transformation	TP-09
Comparative Analysis of Inflectional and Derivational Morphology	Morphological Classification, Language Analysis	TP-10

Finite-State Morphological Parsing Techniques	Morphological Parsing, Finite-State Processing	TP-11
Morphological Analysis for Indian Languages	Language-Specific Morphology, NLP Challenges	TP-12
Porter Stemmer: Design, Working Mechanism, and Applications	Stemming Algorithms, Text Preprocessing	TP-13
Stemming vs Morphological Parsing: A Comparative Study	Word Reduction, Morphological Processing	TP-14
Integrated NLP Pipeline: From Regular Expressions to Morphological Processing	End-to-End Language Processing Pipeline	TP-15

Annexure A – Technical Presentation

Team No.	Technical Presentation Title	Description
1	Evolution of Natural Language Processing: From Rule-Based Systems to Modern NLP	Analyze the historical development of NLP models, key milestones, and how language processing has evolved from handcrafted rules to intelligent systems.
2	Comparative Study of NLP Models and Algorithms	Explore different NLP models and algorithms used for language processing, comparing their working principles, strengths, and limitations.
3	Regular Expressions for Real-World Text Processing	Demonstrate how regular expressions are used for data validation, text extraction, log analysis, and information retrieval in practical applications.
4	Designing Efficient Regular Expression Patterns for Information Extraction	Analyze complex regular expression patterns used to extract emails, phone numbers, URLs, dates, and structured information from unstructured text.
5	Finite State Automata in Natural Language Processing	Explain the concepts of deterministic and non-deterministic finite automata and their role in lexical analysis and language recognition.
6	Building a Finite State Automaton for Token Recognition	Design and demonstrate a finite state automaton that recognizes tokens such as identifiers, keywords, and numbers in textual data.
7	Morphology and Word Formation in Natural Languages	Investigate how words are structured and formed using morphemes, and analyze the importance of morphology in NLP systems.
8	Inflectional Morphology: Understanding Grammatical Variations	Examine how inflectional changes affect word forms based on tense, number, gender, and case across different languages.
9	Derivational Morphology and Vocabulary Expansion	Analyze derivational processes that create new words and study their impact on semantic interpretation and language understanding.
10	Comparative Analysis of Inflectional and Derivational Morphology	Compare the characteristics, applications, and challenges of inflectional and derivational morphology in NLP tasks.
11	Finite-State Morphological Parsing Techniques	Explore finite-state transducers and parsing techniques used to analyze and generate morphological structures in natural languages.
12	Morphological Analysis for Indian Languages	Study the challenges and solutions involved in performing morphological processing for morphologically rich Indian languages such as Tamil, Hindi, and Telugu.
13	Porter Stemmer: Design, Working Mechanism, and Applications	Analyze the Porter Stemming algorithm, its rule-based approach, implementation steps, and role in information retrieval systems.
14	Stemming vs Morphological Parsing: A Comparative Study	Compare stemming and morphological parsing techniques based on accuracy, computational complexity, and application scenarios.
15	Integrated NLP Pipeline: From Regular Expressions to Morphological Processing	Demonstrate a complete NLP preprocessing pipeline incorporating regular expressions, finite state automata, stemming, and morphological analysis for text processing.

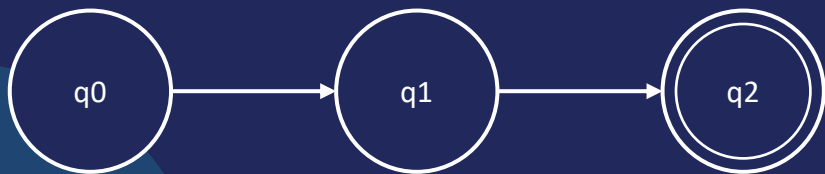
Rubrics for Calculation

Technical Presentation					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Team No. / Criteria Level	Technical Content Accuracy	Problem Identification	Use of Tools	Communication	Professionalism	Total %
13	22	19	20	15	10	86

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurbaran T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

FINITE STATE AUTOMATA



Modeling language with states and transitions

What Is a Finite State Automaton?

A finite state automaton (FSA) is an abstract machine that recognizes patterns by moving between a finite set of states as it reads input symbols one at a time.

1 Q — States

A finite set of configurations the machine can be in

2 Σ — Alphabet

The finite set of input symbols (letters, phonemes, tags)

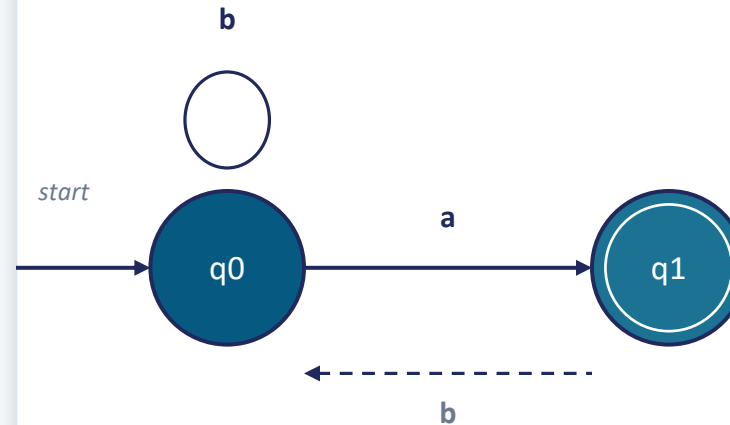
3 δ — Transition Function

Rule for moving from one state to another on a symbol

4 q_0 & F — Start & Accept States

Where reading begins, and states that signal a valid string

Example: strings of a's and b's ending in "a"



q_1 (double circle) is the accepting state — reached only when the last symbol read is "a".

Types of Finite State Automata

NLP systems rely on a few key variants, each suited to a different kind of task.

DFA

Deterministic FSA

Exactly one transition per state/symbol pair.
Fast, unambiguous matching — ideal for tokenizers and lexical scanners.

NFA

Non-deterministic FSA

Multiple possible transitions (or none) per symbol. More compact to write; simulated efficiently via subset construction.

FST

Finite State Transducer

Reads AND writes symbols on each transition.
Maps surface forms to underlying forms — the workhorse of morphology.

Applications in NLP

FSAs underpin many efficient, well-understood building blocks of language processing pipelines.

1 Tokenization

Splitting raw text into words, punctuation, and sentence boundaries using pattern-matching automata.

2 Morphological Analysis

Parsing word forms like "running" → run + ING with finite state transducers.

3 Spelling & Grammar Checking

Matching input against a dictionary automaton to flag or correct invalid word forms.

4 Named Entity Recognition

Chunking sequences such as dates, phone numbers, and named entities via pattern-based FSAs.

5 Speech Recognition

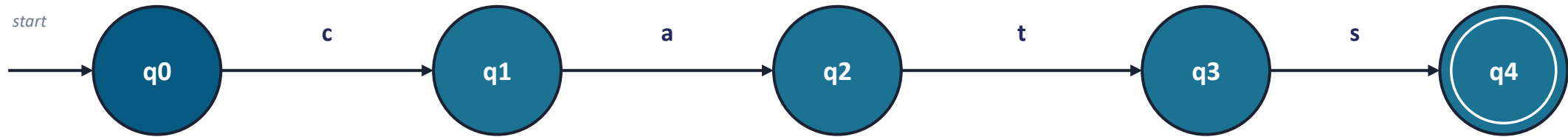
Modeling phoneme sequences and pronunciation lexicons as weighted finite state transducers.

6 Regular Expression Engines

Compiling regex patterns into equivalent automata for fast text search and matching.

Worked Example: Recognizing English Plurals

An FSA that accepts a noun stem optionally followed by the plural suffix “-s” (e.g. cat, cats).



q3 is also an accepting state (singular “cat”) — the “s” transition to q4 accepts the plural “cats”.

Key takeaway

Even simple automata like this one scale to model complex morphological and syntactic patterns across entire languages.

Strengths & Limitations of FSAs

Knowing what finite state automata are good at — and where they run out of power.



Strengths

● Fast, predictable matching

Linear-time scanning with no backtracking — ideal for high-throughput pipelines.

● Compact & minimizable

DFA minimization collapses equivalent states into the smallest possible machine.

● Mathematically well-understood

Closure under union, concatenation, and Kleene star supports reliable composition.

● Composable pipelines

Transducers (FSTs) chain together, letting complex processing emerge from simple parts.



Limitations

● Cannot count

No memory of how many symbols have been seen, so balanced structures are out of reach.

● No recursion

Center-embedded clauses (“the rat the cat chased ran”) exceed what a finite state model can track.

● Regular languages only

Full syntactic parsing needs context-free grammars, one level up the hierarchy.

● State explosion

Some phenomena (e.g. long-distance agreement) require impractically many states.

Key Takeaways & Further Reading

The essentials to carry forward, and where to go deeper.

Key Takeaways

1 States, symbols, and transitions

An FSA is fully defined by $(Q, \Sigma, \delta, q_0, F)$ — a small, precise toolkit.

2 Three key variants

DFA, NFA, and FST cover deterministic matching, compact ambiguity, and read/write mapping.

3 Everywhere in the pipeline

Tokenizers, morphology, spelling checkers, NER, ASR, and regex engines all lean on FSAs.

4 Speed has a price

No counting and no recursion — nested or long-distance structure is out of reach.

5 The base of the hierarchy

Regular languages are the foundation more expressive grammars build on.

Further Reading

1 Speech and Language Processing

Jurafsky & Martin — chapters on regular expressions & automata

2 Introduction to Automata Theory, Languages, and Computation

Hopcroft, Motwani & Ullman

3 Finite-State Language Processing

Roche & Schabes (eds.)

4 OpenFst & Foma

Open-source toolkits for building and composing finite state transducers



What Is a Finite State Automaton?

A finite state automaton (FSA) is an abstract machine that recognizes patterns by moving between a finite set of states as it reads input symbols one at a time.

1. **Q — States**
A finite set of configurations the machine can be in.
2. **Σ — Alphabet**
The set of input symbols (letters, digits, etc.).
3. **δ — Transition Function**
A function that takes a state and an input symbol and returns a new state.
4. **q0 & f — Start & Accept States**
q0 is the start state. f is the set of accepting states.

Example: strings of a's and b's ending in "a"



q1 is the only state in the accepting set — reached only when the last symbol read is 'a'.

Mevalurkuppam, Tamil Nadu,
India

Cse & It Department, Mevalurkuppam, Tamil Nadu
602105, India
Lat 13.026065° Long 80.016836°
Thursday, 16/07/2026 10:40 AM GMT +05:30